

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Прикладная разработка на java
на тему: Разработка десктоп аудиоплеера с базовыми функциями

Руководитель

(уч. степень, звание)

Метелкин А. С.
(фамилия, инициалы)

(подпись) (дата)

Студент ИС-123
(группа)

Осетров Е. В.
(фамилия, инициалы)

(подпись) (дата)

Члены комиссии

(подпись) (Ф.И.О.)

(подпись) (Ф.И.О.)

Муром 2026

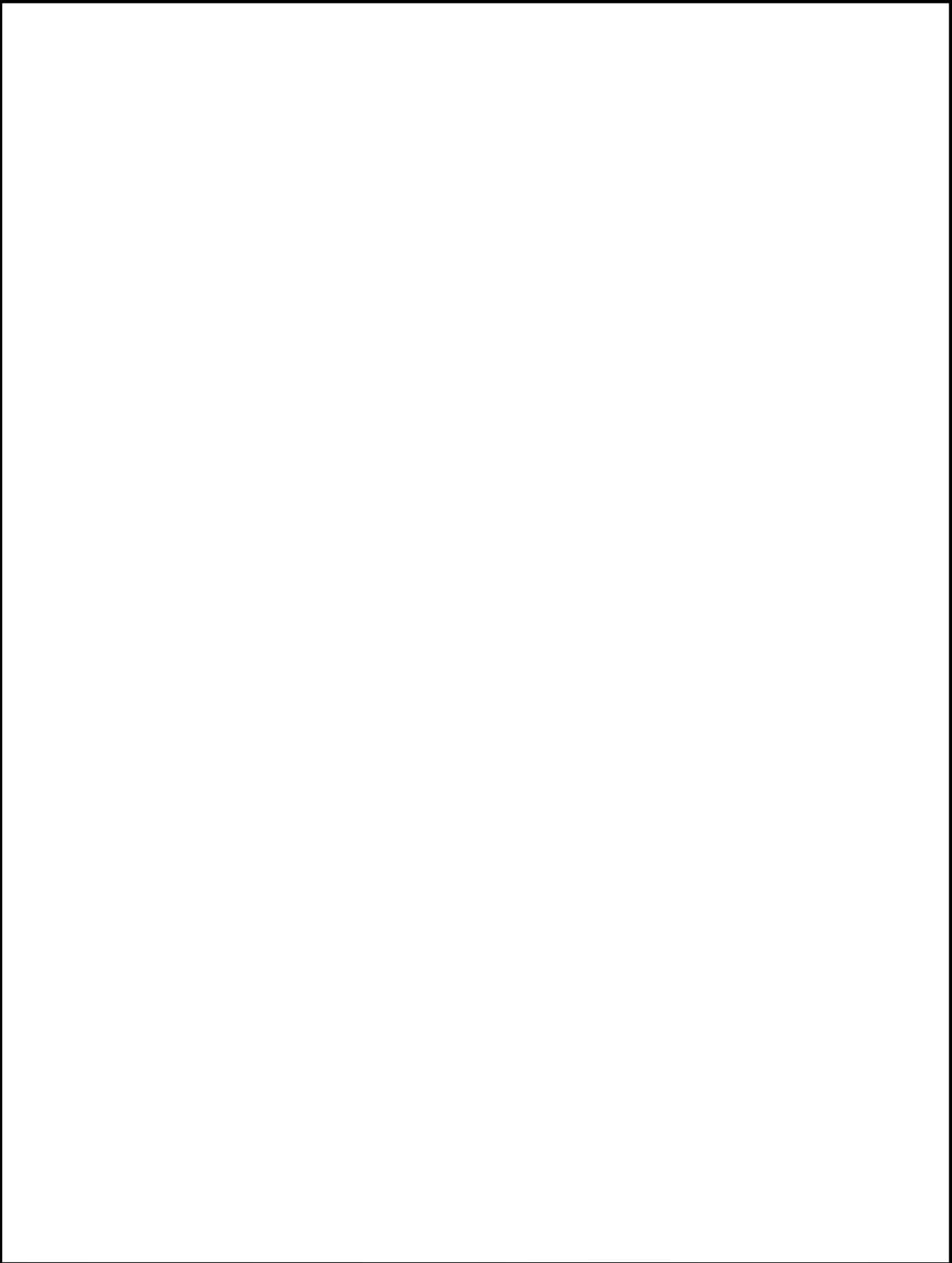


В курсовой работе отражена разработка настольного приложения PulseBeat для прослушивания музыкальных композиций, управления медиатекой, добавления избранных треков и публикации пользовательских отзывов. Программа реализована на языке Java с применением JavaFX, Maven, JDBC, Firebird/Red Database, BCrypt и JAudioTagger. В проекте спроектирована модульная архитектура на основе принципов объектно-ориентированного программирования, интерфейсов и паттернов проектирования Repository, Strategy, Factory, Decorator и Chain of Responsibility. Разработан графический интерфейс, позволяющий выполнять вход и регистрацию, импортировать MP3-файлы, читать метаданные, искать треки, управлять очередью воспроизведения, включать случайный порядок и повтор, редактировать сведения о композициях и работать с отзывами. Проведено тестирование основных сценариев работы приложения и проверена корректность взаимодействия с базой данных.

Содержание

ВВЕДЕНИЕ	5
1 Анализ технического задания.....	7
1.1 Анализ существующих аналогов.....	8
1.2 Выбор технологий разработки.....	9
1.3 Использование паттернов проектирования	10
2 Проектирование программы.....	12
2.1 Проектирование системы на Java	13
2.2 Проектирование системы хранения данных	14
2.3 Проектирование графического интерфейса	16
3 Разработка приложения	19
3.1 Разработка системы на Java	19
4 Тестирование	25
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

					МИВУ.09.03.02 019 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Осетров Е. В.			Разработка десктоп аудиоплеера с базовыми функциями	Лит.	Лист	Листов
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								



					МИВУ.09.03.02 019 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка десктоп аудиоплеера с базовыми функциями	Лит.	Лист	Листов
Разраб.		Осетров Е. В.						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

					МИВУ.09.03.02 019 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка десктоп аудиоплеера с базовыми функциями	Лит.	Лист	Листов
Разраб.		Осетров Е. В.						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

ВВЕДЕНИЕ

В современных условиях музыкальные сервисы и настольные медиаплееры остаются востребованными программными решениями, поскольку пользователю необходимо не только воспроизводить аудиофайлы, но и удобно организовывать личную медиатеку. При большом количестве композиций важными становятся быстрый поиск, корректное чтение метаданных, выделение избранных треков, хранение пользовательских оценок и возможность простого импорта файлов из локальных папок.

Разработка музыкального плеера позволяет рассмотреть несколько важных аспектов программирования: построение графического интерфейса, работу с базой данных, обработку файловой системы, воспроизведение мультимедиа, защиту пользовательских паролей и применение паттернов проектирования. В рамках данной работы разрабатывается приложение PulseBeat, ориентированное на работу с локальными MP3-файлами и хранение информации о медиатеке в базе данных.

Целью курсового проекта является разработка настольного приложения для управления музыкальной библиотекой и воспроизведения аудиотреков с использованием JavaFX и базы данных Firebird/Red Database.

Для достижения поставленной цели необходимо решить следующие задачи:

- спроектировать архитектуру приложения с использованием интерфейсов и паттернов проектирования;
- разработать систему авторизации и регистрации пользователей;
- реализовать импорт MP3-файлов, чтение метаданных и сохранение сведений о треках;
- создать графический интерфейс для медиатеки, очереди воспроизведения, избранного и отзывов;
- реализовать воспроизведение треков, управление громкостью, перемотку, повтор и случайный порядок;
- провести тестирование основных функций программы.

Объектом исследования является процесс организации локальной музыкальной медиатеки. Предметом исследования являются программные средства хранения, поиска, воспроизведения и оценки музыкальных композиций. Практическая значимость работы заключается в создании приложения, которое может использоваться как учебный пример комплексной Java-системы с графическим интерфейсом, базой данных и мультимедийной логикой.

					МИВУ.09.03.02 019 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

1 Анализ технического задания

Разработка музыкального плеера требует учета особенностей предметной области, связанной с локальными аудиофайлами, пользовательскими данными, метаданными треков и мультимедийным воспроизведением. Приложение должно быть удобным для пользователя и одновременно достаточно гибким с точки зрения внутренней архитектуры, чтобы отдельные подсистемы можно было расширять без полного изменения программы.

Основными пользователями системы являются слушатели, которым необходимо импортировать музыку из папок, просматривать список композиций, искать нужный трек, добавлять композиции в избранное, оставлять отзывы и воспроизводить выбранные файлы. Поэтому интерфейс должен быть наглядным, а работа с базой данных должна обеспечивать сохранение информации между запусками приложения.

Функциональные требования к приложению включают регистрацию и вход пользователя, хранение учетных записей, импорт MP3-файлов, чтение названия, исполнителя и альбома, отображение медиатеки, поиск по названию и исполнителю, воспроизведение выбранного трека, управление очередью, добавление избранного, создание отзывов и расчет среднего рейтинга.

Нефункциональные требования включают модульность, понятную структуру классов, использование объектно-ориентированных принципов, защиту паролей с помощью хеширования, корректную обработку ошибок подключения к базе данных и удобство сопровождения программного кода.

При формировании требований учитывалось, что музыкальный плеер работает с локальными файлами пользователя. Поэтому приложение не должно зависеть от сетевых сервисов, кроме подключения к локальной или серверной базе данных. Все сведения о треках должны сохраняться в базе, а сами аудиофайлы остаются на диске пользователя и воспроизводятся по сохраненному пути.

					МИВУ.09.03.02 019 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

Для удобства использования приложение должно поддерживать импорт не только отдельного файла, но и целого каталога. Такой сценарий является более естественным для пользователя, поскольку музыкальная коллекция обычно хранится папками. При сканировании каталога необходимо пропускать неподдерживаемые файлы и не прерывать весь процесс из-за одного ошибочного элемента.

Отдельным требованием является возможность восстановления понятных сведений о треке. В музыкальных файлах часто отсутствуют теги или они заполнены некорректно. Поэтому программа должна иметь резервный способ определения названия и исполнителя. В проекте для этого используется анализ имени файла.

Система отзывов и рейтингов расширяет обычную медиатеку до личного каталога впечатлений. Пользователь может не только слушать композиции, но и сохранять собственную оценку. Это позволяет показать работу со связанными таблицами и агрегирующими запросами к базе данных.

Техническое задание предполагает, что приложение должно быть расширяемым. Например, в будущем можно добавить поддержку других аудиоформатов, экспорт плейлистов, сортировку по жанрам, подключение к внешним источникам метаданных или новый режим воспроизведения. Поэтому архитектура не должна быть жестко привязана к одному классу контроллера или одному способу хранения данных.

1.1 Анализ существующих аналогов

В качестве аналогов можно рассмотреть VLC Media Player, AIMP, Windows Media Player, Foobar2000 и музыкальные разделы крупных потоковых сервисов. Указанные решения обладают развитым функционалом воспроизведения, поддерживают плейлисты, сортировку, работу с библиотекой и настройку интерфейса.

					МИВУ.09.03.02 019 ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

VLC Media Player известен широкой поддержкой форматов и стабильностью воспроизведения. AIMP ориентирован на удобную локальную медиатеку и настройку внешнего вида. Foobar2000 отличается высокой гибкостью и возможностью расширения. Поточковые сервисы, например Яндекс Музыка или Spotify, делают акцент на рекомендациях и облачном хранении, но требуют сетевого подключения и не всегда удобны для локальной коллекции.

1.2 Выбор технологий разработки

В качестве основного языка программирования выбран Java 17. Данный язык поддерживает объектно-ориентированное программирование, имеет устойчивую экосистему библиотек и хорошо подходит для создания настольных приложений. Для сборки и управления зависимостями используется Maven, что упрощает подключение библиотек и запуск проекта.

Графический интерфейс реализован на JavaFX 21. Данная технология предоставляет готовые элементы управления, поддержку CSS-стилей, работу с анимацией и мультимедийные возможности через модуль JavaFX Media. Для воспроизведения аудио используется класс MediaPlayer, позволяющий управлять запуском, паузой, позицией трека и громкостью.

Для хранения данных используется Firebird/Red Database через JDBC-драйвер Jaybird. Такой подход позволяет сохранять пользователей, треки, избранные записи и отзывы. Пароли пользователей хранятся не в открытом виде, а в виде BCrypt-хеша. Для чтения ID3-тегов MP3-файлов применяется библиотека JAudioTagger.

Выбор JavaFX связан с тем, что библиотека позволяет создавать современный интерфейс без перехода к web-технологиям. В JavaFX можно программно формировать элементы, назначать обработчики событий, использовать CSS-оформление и подключать мультимедийные возможности. Это соответствует задаче разработки настольного плеера.

					МИВУ.09.03.02 019 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подпись	Дата		

Maven применяется не только как средство сборки, но и как способ фиксировать зависимости проекта. В pom.xml указаны версии JavaFX, Jaybird, JNA, BCrypt и JAudioTagger. Это упрощает перенос проекта на другой компьютер, поскольку разработчику не нужно вручную искать необходимые jar-файлы.

Использование JDBC делает слой работы с базой данных универсальным. DAO-классы не зависят от графического интерфейса и выполняют только операции чтения и записи. Если потребуется заменить базу данных, основные изменения будут сосредоточены в настройках подключения и SQL-запросах.

Библиотека BCrypt выбрана потому, что хранение паролей в открытом виде является недопустимым даже в учебном приложении. BCrypt автоматически использует соль и делает проверку пароля через сравнение хеша. Это демонстрирует правильный подход к работе с учетными данными.

JAudioTagger решает задачу чтения ID3-тегов. Без такой библиотеки пришлось бы вручную разбирать бинарную структуру MP3-файла, что выходит за рамки задачи. Подключение готовой библиотеки позволяет сосредоточиться на интеграции метаданных в медиатеку.

1.3 Использование паттернов проектирования

Важным требованием проекта является применение паттернов проектирования. В PulseBeat используется несколько паттернов, каждый из которых решает отдельную архитектурную задачу и уменьшает связанность между классами.

Паттерн Repository применяется в интерфейсе TrackRepository. Контроллер работает не с конкретным DAO-классом, а с абстракцией репозитория. Благодаря этому можно заменить способ хранения треков или добавить кэширование без изменения логики интерфейса.

Паттерн Decorator используется в CachedTrackRepository. Данный класс оборачивает другой репозиторий и добавляет кэширование списка треков. Это

уменьшает количество обращений к базе данных и сохраняет общий контракт TrackRepository.

Паттерн Strategy используется в PlaybackOrderStrategy. Разные реализации выбирают следующий трек по-разному: последовательно, случайно или с повтором текущей композиции. Контроллер получает нужную стратегию через фабрику и не содержит сложной логики выбора внутри каждого сценария воспроизведения.

Паттерн Chain of Responsibility применяется при чтении метаданных. FallbackTrackMetadataReader последовательно пробует TagTrackMetadataReader и FileNameTrackMetadataReader. Если теги отсутствуют или не читаются, программа получает данные из имени файла.

Применение паттернов в проекте не является формальным. Каждый паттерн закрывает конкретную проблему. Например, Strategy избавляет контроллер от необходимости хранить несколько вариантов перехода к следующему треку. Контроллер только запрашивает стратегию и вызывает общий метод.

Repository и DAO используются совместно. DAO знает о таблицах и SQL-запросах, а репозиторий предоставляет более общий интерфейс для работы с треками. Благодаря этому в контроллере меньше деталей, связанных с базой данных.

Decorator особенно полезен для кэширования. CachedTrackRepository не изменяет поведение исходного репозитория с точки зрения внешнего контракта, но добавляет хранение списка треков в памяти. При изменении данных кэш сбрасывается или обновляется.

Factory упрощает создание зависимостей. Если бы объекты создавались прямо в контроллере, MainController был бы еще сильнее связан с конкретными реализациями. Фабрика собирает рабочую конфигурацию в одном месте.

Chain of Responsibility делает импорт музыки устойчивым. Если первый способ чтения метаданных не дал результата, программа переходит к следующему. Пользователь при этом получает добавленный трек, а не ошибку импорта.

					МИВУ.09.03.02 019 ПЗ	Лист
						11
Изм.	Лист	№ докум.	Подпись	Дата		

2 Проектирование программы

на этапе проектирования программа была разделена на несколько логических уровней: уровень представления, уровень контроллеров, сервисный уровень, уровень репозитория и dao, уровень моделей, а также утилитарные классы. такое разделение делает проект понятным и упрощает внесение изменений.

Главной точкой входа является MainApp. После запуска создается окно JavaFX и отображается экран авторизации. Если пользователь успешно входит в систему, сцена переключается на основной интерфейс медиатеки. Контроллеры отвечают за построение элементов интерфейса, реакцию на действия пользователя и передачу запросов в слой данных. На рисунке 1 представлена общая диаграмма этапов разработки программы визуализации алгоритмов сортировки.

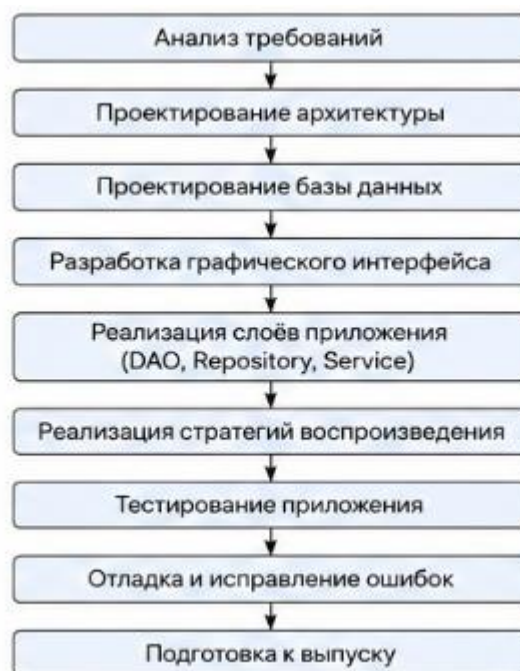


Рисунок 1 – Диаграмма этапов разработки программы

2.1 Проектирование системы на Java

Структура проекта построена по пакетам. Пакет `controller` содержит классы `LoginController` и `MainController`, отвечающие за экраны приложения. Пакет `dao` включает классы доступа к базе данных: `UserDAO`, `TrackDAO`, `FavoriteDAO` и `ReviewDAO`. Пакет `model` содержит сущности `User`, `Track`, `Favorite` и `Review`. Пакет `repository` отвечает за работу с треками на уровне абстракций. Пакеты `metadata` и `playback` содержат интерфейсы и реализации стратегий.

В проекте предусмотрены интерфейсы `TrackRepository`, `TrackMetadataReader` и `PlaybackOrderStrategy`. Наличие интерфейсов позволяет программировать на уровне абстракций и подменять реализации без изменения контроллеров. Например, `SqlTrackRepository` можно заменить на `ReadOnlyTrackRepository`, если приложению нужно запретить изменение списка треков.

```
public interface PlaybackOrderStrategy {  
    int nextIndex(int currentIndex, int size);  
    int previousIndex(int currentIndex, int size);  
}
```

Отдельного внимания заслуживает `ApplicationFactory`. Фабрика создает репозиторий, читатель метаданных и стратегии воспроизведения. За счет этого `MainController` не собирает зависимости вручную и остается сосредоточенным на логике интерфейса.

Модель `Track` содержит идентификатор, название, исполнителя, альбом и путь к файлу. Такой набор полей достаточен для отображения карточки трека и запуска воспроизведения. Метод `toString` возвращает строку с исполнителем и названием, что удобно при выводе трека в списках.

Модель `User` содержит идентификатор, логин и хеш пароля. В интерфейсе пароль после входа не используется напрямую, однако хранение хеша в объекте позволяет сохранить соответствие данным из базы. В реальном приложении это поле можно было бы дополнительно скрыть.

Модель Review хранит идентификатор отзыва, идентификатор пользователя, идентификатор трека, альбом, оценку, текст и дату. Наличие даты позволяет сортировать отзывы по времени добавления и показывать пользователю наиболее новые записи первыми.

Сервис PlayerService выделен для логики, связанной с воспроизведением. Даже если часть управления медиаплеером находится в MainController, наличие отдельного сервиса показывает направление дальнейшего разделения ответственности.

Класс MusicScanner предназначен для поиска музыкальных файлов в каталоге. Его можно развивать отдельно от интерфейса, добавляя фильтрацию расширений, ограничение глубины обхода или обработку ошибок доступа к отдельным папкам.

2.2 Проектирование системы хранения данных

Для хранения данных используется реляционная база данных. В приложении выделены четыре основные сущности: пользователь, трек, избранное и отзыв. Пользователь хранит учетные данные, трек хранит сведения о музыкальном файле, избранное связывает пользователя и трек, отзыв содержит оценку и текстовую рецензию.

Подключение к базе данных выполняется в DatabaseUtil. Параметры соединения выносятся в файл application.properties. При необходимости можно указать полный JDBC URL.

```
db.host=localhost
db.port=3051
db.path=test
db.user=SYSDBA
db.password=masterkey
```

DAO-классы используют PreparedStatement, что позволяет передавать значения параметров отдельно от SQL-текста. Такой подход делает запросы понятнее и снижает риск ошибок при работе со строковыми значениями.

					МИВУ.09.03.02 019 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подпись	Дата		

При проектировании базы данных важно обеспечить целостность связей. Таблица Favorites фактически является связующей таблицей между Users и Tracks. Одна и та же композиция может быть избранной у разных пользователей, а один пользователь может добавить несколько композиций.

Таблица Reviews также связана с пользователями и треками. Поле rating хранит числовую оценку, а поле review_text - текст комментария. Поле review_date заполняется текущей датой и временем при добавлении записи, что позволяет сортировать отзывы без дополнительной логики на стороне приложения.

Таблица Tracks хранит путь к файлу. Это важное поле, поскольку сам аудиофайл не помещается в базу данных. Такой подход экономит место и упрощает воспроизведение через JavaFX MediaPlayer, который может работать с URI локального файла.

Возможным улучшением базы данных является добавление ограничений уникальности. Например, для Users можно сделать уникальным поле login, а для Tracks - file_path. Это защитит данные от дублей даже в случае ошибки на уровне приложения.

Еще одним направлением развития является добавление таблиц Playlists и PlaylistTracks. Тогда пользователь сможет создавать собственные плейлисты. Текущая архитектура допускает такое расширение, поскольку работа с треками уже вынесена в отдельный слой.

Схема системы хранения данных, представленная на рисунке 2.

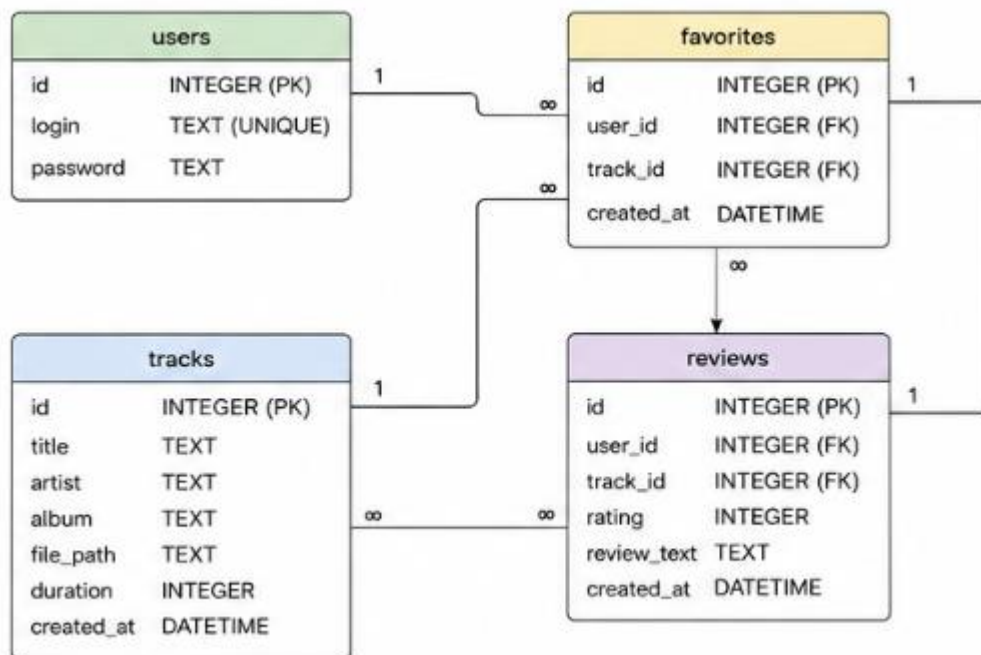


Рисунок 2 – Схема системы хранения данных

2.3 Проектирование графического интерфейса

Графический интерфейс проектируется как настольное приложение с экраном входа и основным рабочим пространством. Экран входа содержит поля логина и пароля, кнопку авторизации, кнопку регистрации и сообщение об ошибке или результате действия. После успешной авторизации пользователь попадает в медиатеку.

Основной экран состоит из верхней панели окна, боковой панели навигации, центральной области с библиотекой, правой панели с очередью и нижней панели плеера. В центральной области отображаются карточки треков, статистика медиатеки, строка поиска и кнопки импорта. Правая панель помогает видеть текущую очередь и информацию о выбранной композиции.

Верхняя панель: бренд, свернуть, развернуть, закрыть

Левая панель: медиатека, избранное, импорт, выход

Центр: поиск, статистика, карточки треков, страница трека

Правая панель: текущий трек, очередь, рейтинг

Нижняя панель: назад, play/pause, вперед, shuffle, repeat, громкость,

прогресс

При проектировании интерфейса учитывалось, что пользователь должен видеть основные действия без перехода по сложным меню. Поэтому кнопки импорта, избранного, воспроизведения и поиска доступны прямо на главном экране.

Строка поиска работает как быстрый фильтр. Пользователю не нужно нажимать отдельную кнопку, поскольку список треков обновляется после изменения текста. Такой подход делает работу с большой медиатекой быстрее.

Карточки треков позволяют визуально отделить одну композицию от другой. В карточке показываются основные сведения и действия. При выборе трека пользователь может перейти к расширенной странице с отзывами и рейтингом.

Правая панель с очередью делает воспроизведение предсказуемым. Пользователь видит, какой трек выбран сейчас и какие композиции находятся рядом. Это особенно важно при включенном случайном порядке.

Нижняя панель плеера должна оставаться доступной независимо от того, какой раздел открыт в центре. Поэтому она размещена в нижней части общей компоновки и не заменяется при переходе между медиатекой и страницей трека.

Оформление интерфейса вынесено в файл `styles.css`. Это позволяет разделить программную логику и визуальное представление. Контроллеры присваивают элементам CSS-классы, а внешний вид задается стилями: цвета, отступы, радиусы, тени, фоновые эффекты и состояние кнопок.

Схема работы графического интерфейса представлена на рисунке 3.

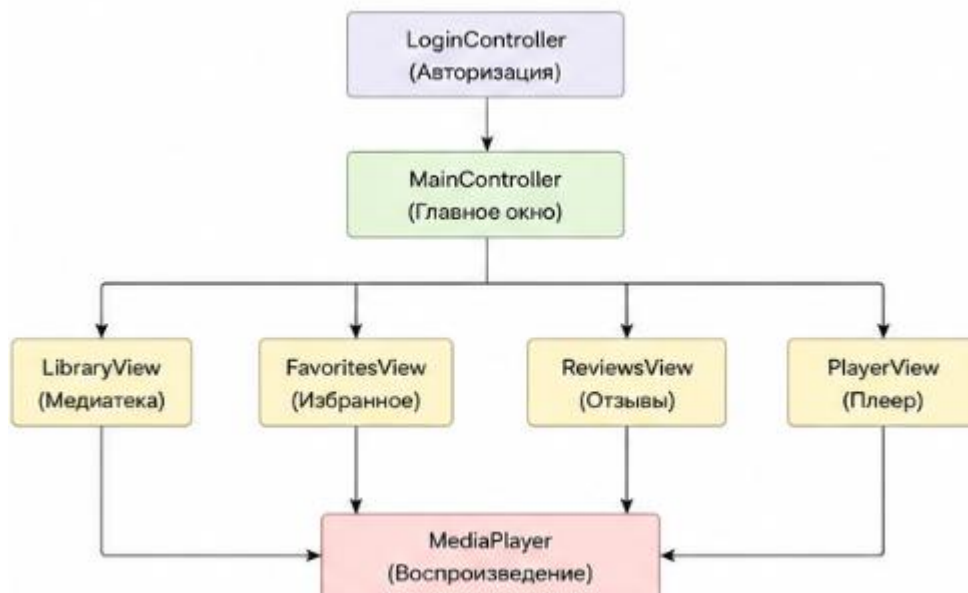


Рисунок 3 – Схема работы графического интерфейса

3 Разработка приложения

Схема спроектированной системы на Java представлена на рисунке 4.

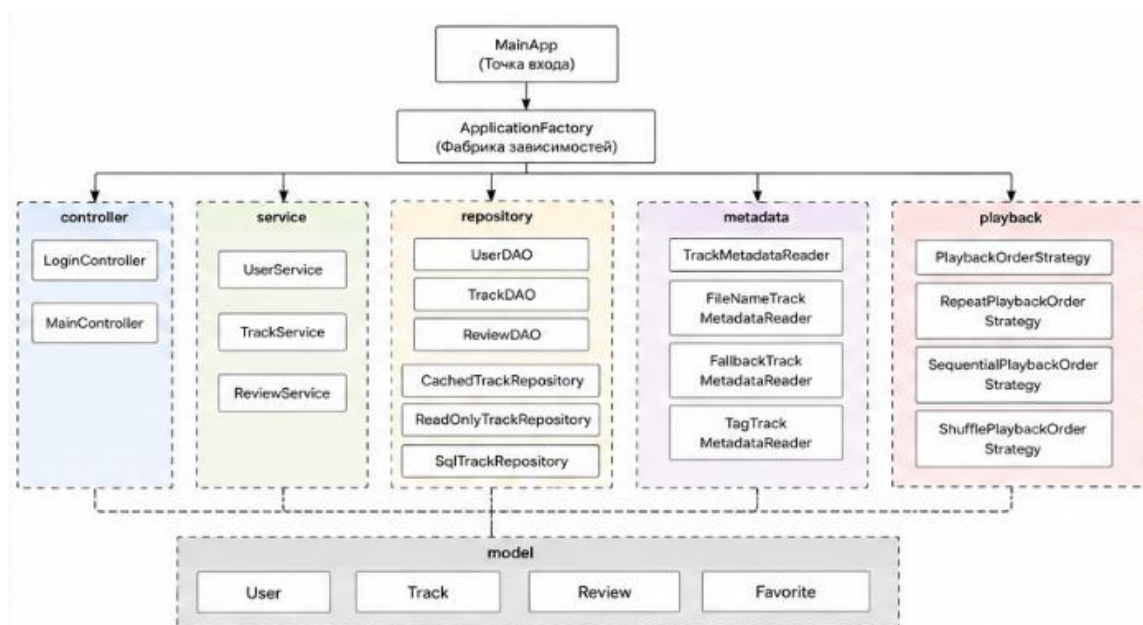


Рисунок 4 – Схема спроектированной системы на Java

3.1 Разработка системы на Java

Разработка десктопного аудиоплеера выполнялась поэтапно с постепенным расширением функциональности и усложнением архитектуры программы. Основной целью разработки являлось создание музыкального приложения, поддерживающего воспроизведение аудиофайлов, хранение информации о треках, управление пользовательскими данными, а также возможность добавления музыкальных композиций в избранное и публикации рецензий. В процессе реализации особое внимание уделялось разделению ответственности между модулями, независимости компонентов и использованию объектно-ориентированного подхода.

На начальном этапе разработки была реализована базовая структура проекта. Сначала были созданы основные модели данных, описывающие сущности системы. Главной задачей данного этапа стало формирование структуры хранения информации о пользователях, музыкальных композициях,

избранных треках и пользовательских рецензиях. После определения структуры данных были реализованы основные классы моделей и методы доступа к их полям.

После реализации базовой структуры данных была разработана подсистема хранения информации о музыкальных композициях. Основой данной подсистемы стал класс Track, определяющий структуру хранения информации о треках. Класс содержит идентификатор композиции, название трека, исполнителя, альбом и путь к аудиофайлу. Также в классе был переопределён метод toString(), обеспечивающий удобное отображение треков в пользовательском интерфейсе в формате «исполнитель – название композиции». Класс Track представлен на рисунке 5.

```
public class Track { 33 usages
    private int id; 3 usages
    private String title; 4 usages
    private String artist; 4 usages
    private String album; 3 usages
    private String filePath; 3 usages

    public Track() {} no usages

    public Track(int id, String title, String artist, String album, String filePath) {
        this.id = id;
        this.title = title;
        this.artist = artist;
        this.album = album;
        this.filePath = filePath;
    }
}
```

Рисунок 5 – Интерфейс Track

После реализации модели музыкальных композиций была разработана подсистема пользователей. Для хранения информации об учётных записях был создан класс User, содержащий идентификатор пользователя, логин и хеш пароля. Использование отдельной модели пользователей позволило подготовить основу для авторизации и персонализации данных внутри приложения. Класс User представлен на рисунке 6.

```

public class User { 8 usages

    private int id; 3 usages
    private String login; 3 usages
    private String passwordHash; 3 usages

    public User() {} no usages

    public User(int id, String login, String passwordHash) {
        this.id = id;
        this.login = login;
        this.passwordHash = passwordHash;
    }
}

```

Рисунок 6 – Класс User

Следующим этапом стала реализация системы пользовательских рецензий. Для этого был создан класс Review, предназначенный для хранения отзывов пользователей о музыкальных композициях и альбомах. Класс содержит идентификатор рецензии, идентификатор пользователя, идентификатор трека, название альбома, оценку, текст рецензии и дату публикации. Во время разработки особое внимание уделялось возможности хранения рецензий как для отдельных треков, так и для музыкальных альбомов. Для этого поле trackId может принимать значение null. Класс Review представлен на рисунке 7.


```

public class Review { 9 usages

    private int id; 3 usages
    private int userId; 3 usages
    private Integer trackId; // может быть null 3 usages
    private String album; // может быть null 3 usages
    private int rating; 3 usages
    private String reviewText; 3 usages
    private LocalDateTime reviewDate; 3 usages

    public Review() {} 1 usage

    public Review(int id, int userId, Integer trackId, String album, 1 usage
                  int rating, String reviewText, LocalDateTime reviewDate) {
        this.id = id;
        this.userId = userId;
        this.trackId = trackId;
        this.album = album;
        this.rating = rating;
        this.reviewText = reviewText;
        this.reviewDate = reviewDate;
    }
}

```

Рисунок 7 – Класс Review

После реализации системы рецензий была разработана подсистема избранных композиций. Для хранения информации о добавленных пользователем треках был создан класс Favorite. Основной задачей данного класса стало связывание пользователя с выбранной музыкальной композицией. Класс содержит идентификатор пользователя и идентификатор трека, добавленного в избранное. Благодаря отдельной сущности избранного появилась возможность формировать персональный список любимых композиций пользователя. Класс Favorite представлен на рисунке 8.

```

public class Favorite { no usages

    private int userId; 3 usages
    private int trackId; 3 usages

    public Favorite() {} no usages

    public Favorite(int userId, int trackId) {
        this.userId = userId;
        this.trackId = trackId;
    }
}

```

Рисунок 8 – Класс Favorite

После завершения разработки моделей данных была реализована система взаимодействия между компонентами приложения. Пользователь получает доступ к музыкальной библиотеке, выбирает композицию и запускает её воспроизведение. Дополнительно пользователь может добавлять треки в избранное и публиковать рецензии с оценками. Все данные о пользователях, музыкальных композициях и рецензиях обрабатываются через отдельные классы моделей, что обеспечивает независимость компонентов и упрощает сопровождение проекта.

Важной особенностью разработки стало использование объектно-ориентированного подхода. Каждая сущность системы реализована в виде отдельного класса, отвечающего только за хранение и обработку собственных данных. Благодаря этому удалось разделить ответственность между компонентами программы и уменьшить связанность модулей. Такой подход значительно упрощает расширение функциональности приложения и делает архитектуру более понятной.

Дополнительно в процессе разработки была реализована работа с пользовательскими рецензиями и системой избранного. Пользователь может

сохранять понравившиеся композиции и оставлять текстовые отзывы с оценками. Для хранения даты публикации рецензии использовался класс `LocalDateTime`, обеспечивающий удобную работу со временем создания отзывов.

Также в процессе разработки особое внимание уделялось удобству отображения данных в пользовательском интерфейсе. Например, переопределение метода `toString()` в классе `Track` позволило автоматически отображать треки в читаемом формате без необходимости дополнительной обработки строк в интерфейсе приложения.

В результате разработки была создана модульная система десктопного аудиоплеера, основанная на объектно-ориентированном подходе. Архитектура программы включает отдельные модели для хранения информации о пользователях, музыкальных композициях, избранных треках и пользовательских рецензиях. Использование отдельных классов для каждой сущности обеспечило независимость компонентов, упростило сопровождение проекта и позволило подготовить основу для дальнейшего расширения функциональности приложения.

4 Тестирование

Одним из важнейших этапов создания программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные недостатки программы, либо убедиться в ее пригодности для применения.

Целью тестирования является проверка работоспособности программы, то есть правильности выполнения всех функций, описанных выше.

Таблица 1 – Тестирование

№	Действие	Результат
1	2	3
1.	Запуск приложения	Успех, корректное отображение главного окна аудиоплеера
2.	Воспроизведение аудиофайла	Успех, трек воспроизводится без ошибок
3.	Изменение громкости во время воспроизведения	Успех, громкость изменяется корректно
4.	Добавление рецензии к треку	Успех, Рецензия сохраняется и отображается
5.	Поиск трека по названию	Успех, отображаются найденные результаты
6.	Переключение на следующий трек	Успех , начинается воспроизведение следующего трека
7.	Пауза во время воспроизведения	Успех, воспроизведение приостановлено
8.	Возобновление после паузы	Успех, трек продолжает играть
9.	Добавление трека в избранное	Успех, трек добавлен в избранное

Вывод сообщений об ошибках и другие сообщения представлены на рисунках 9-13.

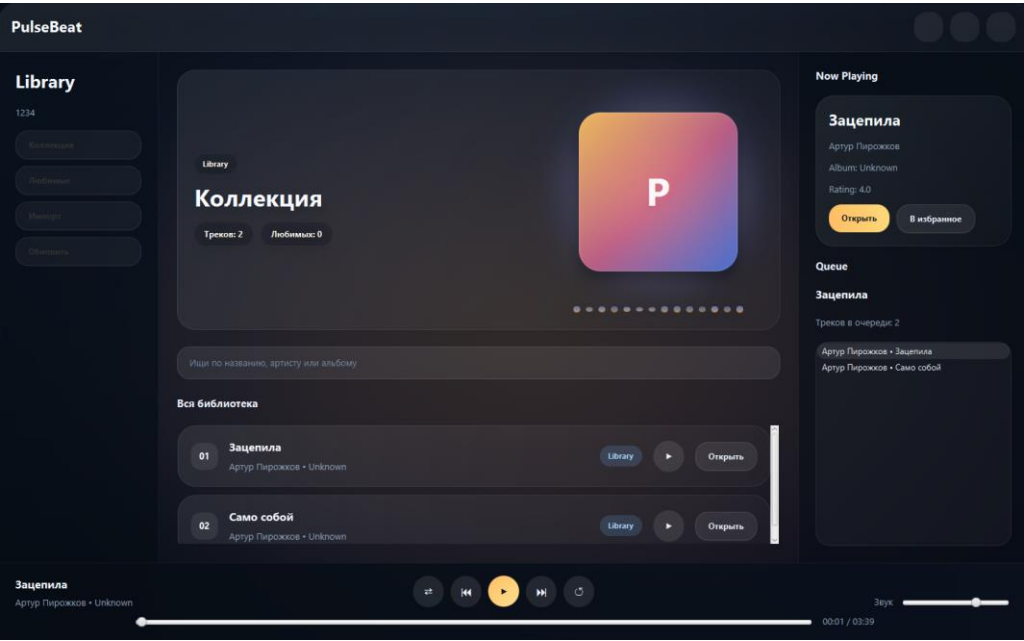


Рисунок 9 – Вывод результата теста №1

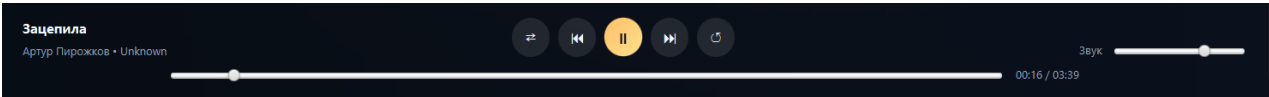


Рисунок 10 – Вывод результата теста №2

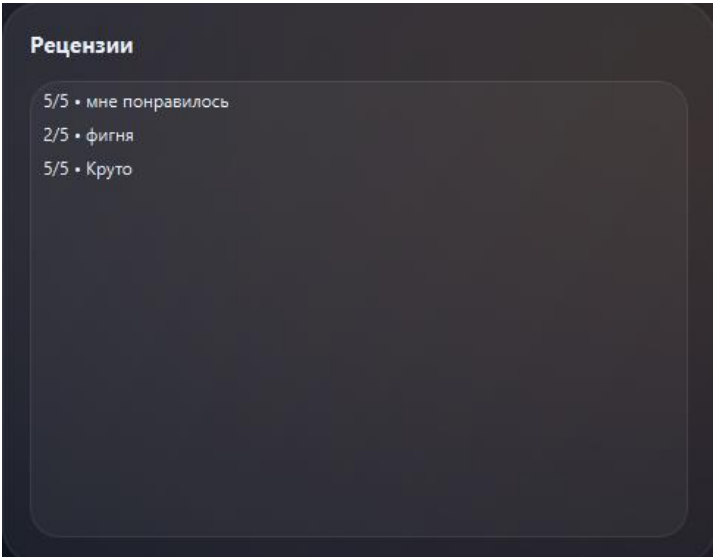


Рисунок 11 – Вывод результата теста №4

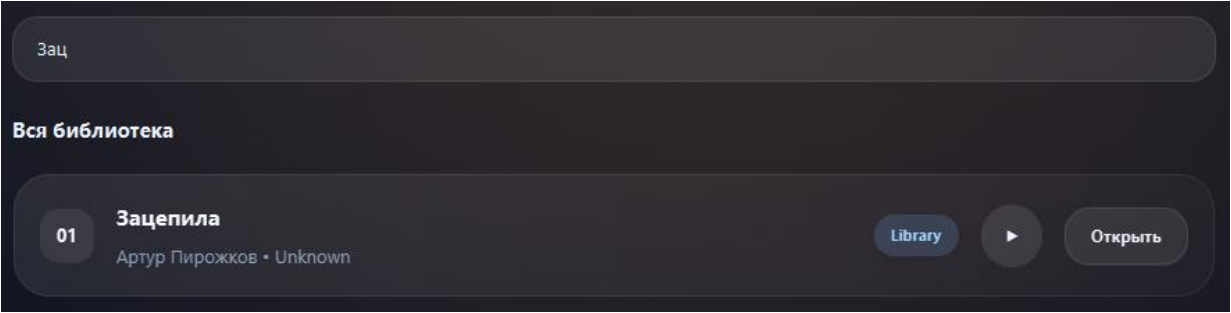


Рисунок 12 – Вывод результата теста №5

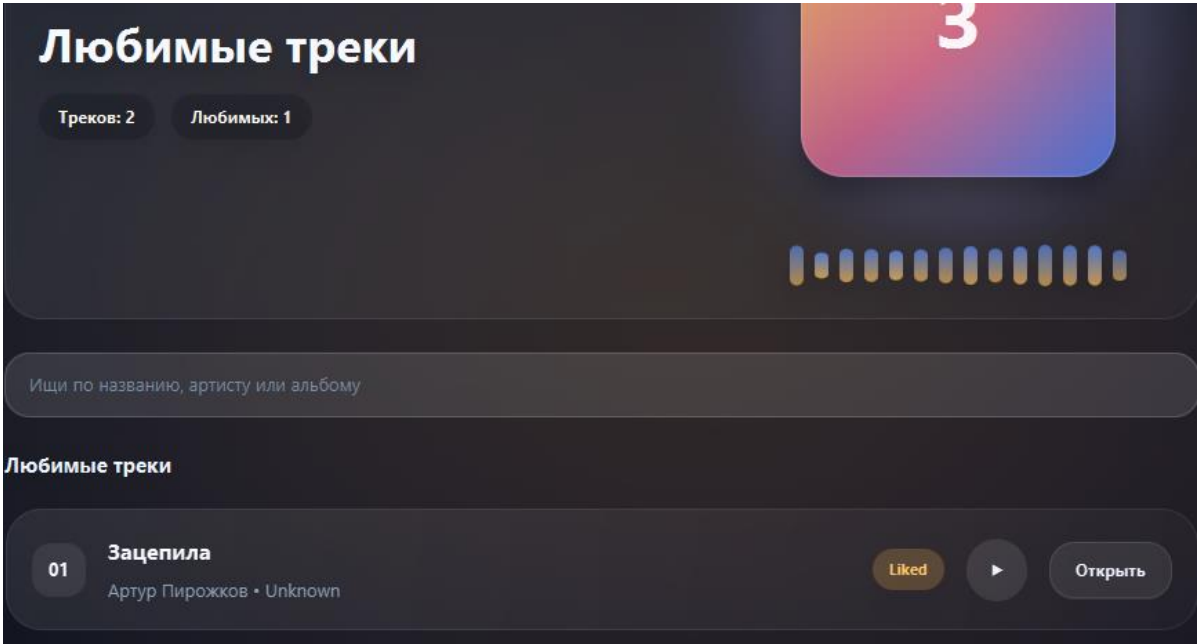


Рисунок 13 – Вывод результата теста №9

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программная система десктопного аудиоплеера на языке Java. Основной целью проекта являлось создание расширяемого приложения, способного воспроизводить музыкальные композиции, хранить информацию о пользователях и треках, а также обеспечивать возможность добавления музыкальных композиций в избранное и публикации пользовательских рецензий.

В процессе разработки была спроектирована модульная архитектура программы, включающая подсистемы хранения музыкальных данных, управления пользователями, обработки рецензий и работы с избранными композициями. Такой подход позволил разделить ответственность между компонентами программы и обеспечить их независимость друг от друга.

В рамках работы были реализованы модели Track, User, Review и Favorite, обеспечивающие хранение и обработку основных данных приложения. Класс Track отвечает за хранение информации о музыкальных композициях, включая название трека, исполнителя, альбом и путь к аудиофайлу. Класс User реализует хранение пользовательских данных, а классы Review и Favorite обеспечивают функциональность рецензий и избранных композиций.

Для взаимодействия компонентов использовался объектно-ориентированный подход, обеспечивающий независимость модулей системы. Данные о пользователях, музыкальных композициях и рецензиях обрабатываются через отдельные классы моделей, а логика работы приложения отделена от хранения информации.

В приложении была реализована возможность хранения пользовательских рецензий с текстовыми отзывами, оценками и датой публикации. Также была добавлена система избранных композиций, позволяющая пользователям формировать собственную музыкальную библиотеку. Для удобства отображения данных в интерфейсе был переопределён метод toString() класса Track, обеспечивающий отображение треков в удобном формате.

При разработке были использованы принципы объектно-ориентированного программирования, обеспечивающие гибкость архитектуры и возможность дальнейшего расширения программы. Разделение функциональности на отдельные классы позволило уменьшить связанность компонентов и упростить сопровождение проекта.

В результате выполнения курсовой работы была создана программная система десктопного аудиоплеера, соответствующая поставленным требованиям и демонстрирующая применение принципов объектно-ориентированного программирования при разработке мультимедийных приложений.

					МИВУ.09.03.02 019 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		29

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Системы управления базами данных [Электронный ресурс] : лабораторный практикум. — Электрон. текстовые данные. — Ставрополь : Северо-Кавказский федеральный университет, 2017. — 148 с. — Режим доступа: <http://www.iprbookshop.ru/75595.html>. — ЭБС «IPRbooks».

2. Лазицкас, Е. А. Базы данных и системы управления базами данных [Электронный ресурс] : учебное пособие / Е. А. Лазицкас, И. Н. Загумённикова, П. Г. Гилевский. — Электрон. текстовые данные. — Минск : Республиканский институт профессионального образования (РИПО), 2018. — 268 с. — Режим доступа: <http://www.iprbookshop.ru/93382.html>. — ЭБС «IPRbooks».

3. Ермаков, А. В. Разработка приложений на языке Java [Электронный ресурс] : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Электрон. текстовые данные. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7. — Режим доступа: <https://www.iprbookshop.ru/150088.html>.

Хорстманн, К. С. Java. Библиотека профессионала. Том 1. Основы [Электронный ресурс] / К. С. Хорстманн, Г. Корнелл. — Электрон. текстовые данные. — Москва : Вильямс, 2019. — 832 с. — Режим доступа: <https://www.litres.ru/key-horstmann/java-biblioteka-professional-a-tom-1-osnovy/>.